

GITHUB COPILOT

Productividad, personalización, SDLC, orquestación y medición de impacto

Casos de uso de la inteligencia artificial para desarrolladores: del autocompletado contextual a los flujos de trabajo orquestados y su medición con datos.

INTRODUCCIÓN

Un asistente integrado en el flujo de trabajo

GitHub Copilot ayuda a escribir código más rápido: se integra con los IDE más populares y ofrece sugerencias contextuales adaptadas al estilo y las preferencias de codificación de cada desarrollador.

Productividad

Acelera tareas rutinarias y ahorra tiempo.

Personalización

Se adapta a las preferencias individuales.

Alcance en el SDLC

Impacta todas las fases del ciclo de vida.

Calidad

Mejora el código y la satisfacción del equipo.

Módulo: casos de uso con GitHub Copilot para la inteligencia artificial de los desarrolladores.

Objetivos de aprendizaje y requisitos previos

Objetivos de aprendizaje

- Identificar cómo Copilot se integra en los flujos de trabajo y se adapta a las preferencias de codificación.
- Explorar su impacto en las distintas fases del SDLC.
- Evaluar las limitaciones de la codificación asistida por IA y su efecto en la eficacia del desarrollo.

Requisitos previos

- Conocimientos básicos de desarrollo de software.
- Familiaridad con al menos un lenguaje de programación.
- Cuenta de GitHub y conocimiento básico de sus funcionalidades.
- Cuenta personal o administrada por una organización para acceder a Copilot.
- Copilot Free es suficiente para aprender.

GitHub Copilot Free: límites y activación

2000

autocompletados de código al mes

50

mensajes de chat al mes

Cómo empezar desde Visual Studio Code

- 1 Abrir Visual Studio Code.
- 2 Seleccionar el icono de GitHub Copilot.
- 3 Hacer clic en "Iniciar sesión para usar GitHub Copilot de forma gratuita".
- 4 Iniciar sesión con la cuenta de GitHub en el navegador.

Educadores, estudiantes y determinados mantenedores de código abierto pueden recibir Copilot Pro de forma gratuita.

Cuatro casos de uso para optimizar el día a día

Aprender lenguajes y marcos

- Ejemplos de funciones y bibliotecas desconocidas
- Transición entre lenguajes
- Sugerencias de API y parámetros
- "Explicar esto" para entender el código

Minimizar el cambio de contexto

- Asistencia nativa en el editor
- Referencias rápidas de API
- Finalización de patrones repetitivos
- Menos consultas externas

Mejorar la documentación

- Comentarios insertados en código complejo
- Descripciones de funciones y parámetros
- Generación de archivos README
- Estilo documental coherente

Automatizar tareas rutinarias

- Código reutilizable (API REST, clases)
- Datos de ejemplo realistas
- Pruebas unitarias
- Traducción y refactorización

De plantillas complejas a historias de usuario

Automatización con plantillas

- Modelos de base de datos, migraciones y configuraciones ORM.
- Puntos de conexión REST con validación, errores y documentación.
- Configuraciones de desarrollo, ensayo y producción.
- Infraestructuras completas de pruebas y microservicios con Docker y CI/CD.

Desde historias de usuario

- Scaffolding de características: modelos, API y front-end.
- Lógica de negocio a partir de reglas en lenguaje natural.
- Patrones de integración: autenticación, registro y servicios externos.
- Calidad integrada: validación, registro y seguridad básica.

Consumo de PRU: 1–2 PRU tareas reutilizables simples • 3–5 PRU andamiaje de proyectos completos • 2–3 PRU por borrador de refactorización en una PR

Acelerar el camino de la PR a producción

Código listo para PR

- Implementaciones completas con manejo de errores, registros y casos límite.
- Coherencia con convenciones y patrones arquitectónicos del proyecto.
- Comentarios, documentación de funciones y actualizaciones del README.
- Pruebas unitarias, de integración y ejemplos de uso.

Revisión inteligente y colaboración

- Detección de problemas antes de crear la PR.
- Comentarios de revisión constructivos y específicos.
- Alternativas cuando los revisores solicitan cambios.
- Resolución de conflictos considerando la intención de ambas ramas.
- Normalización del código y uso compartido de conocimiento del equipo.

Desarrollo con varios agentes especializados

1. Agente de borrador

Analiza requisitos y genera la implementación inicial con errores controlados y pruebas básicas.

2. Agente de revisión

Evalúa calidad, seguridad, rendimiento y cumplimiento de patrones arquitectónicos.

3. Agente de documentación

Genera o actualiza la documentación del código y de las API.

4. Agente de pruebas

Crea conjuntos de pruebas y comprobaciones de calidad.

Cada agente se especializa en un área y contribuye a generar código más completo y preparado para producción.

Cada transferencia consume aproximadamente 1 PRU; un flujo de dos agentes suele utilizar 2–3 PRU.

Razonamiento premium y entrega completa

Razonamiento de alta calidad

- Decisiones arquitectónicas: escalabilidad, mantenimiento y rendimiento.
- Análisis de impacto entre sistemas distribuidos.
- Coordinación de refactorizaciones en múltiples archivos.
- Optimización de patrones de integración.
- Las ejecuciones premium pueden consumir 4 o más PRU por solicitud.

Entrega de extremo a extremo

1. Análisis de requisitos
2. Implementación de la característica
3. Control de calidad y pruebas
4. Documentación
5. Implementación y supervisión

Alineación con las preferencias del desarrollador

Código y pruebas

- Varias sugerencias ante escenarios ambiguos
- Expresiones propias de cada lenguaje
- Casos de prueba, incluidos casos perimetrales
- Códigos auxiliares de documentación

Refactorización

- Reconocimiento de patrones
- Sintaxis moderna del lenguaje
- Coherencia con el estilo del código base

Depuración

- Explicación de errores en lenguaje natural
- Instrucciones de registro para diagnosticar
- Pruebas para aislar errores difíciles

Ciencia de datos

- Funciones y pruebas estadísticas
- Visualización con Matplotlib, Seaborn o Plotly
- Preprocesamiento y escalado de datos
- Evaluación de modelos y métricas

Menos cambio de contexto, más autonomía

Experiencia integrada

- Asistencia nativa en el IDE
- Reconocimiento del contexto del proyecto
- Configuración mínima

Tareas autónomas

- Características de extremo a extremo
- Valores predeterminados inteligentes
- Mejora progresiva del código generado

Enfoque en la calidad

- Procedimientos recomendados integrados
- Coherencia con las convenciones del equipo
- Cobertura de pruebas y documentación

Copilot deja de ser solo un finalizador de código para convertirse en asistencia integrada en todo el proceso de desarrollo.

IA en las fases del ciclo de vida del software

Análisis de requisitos

Prototipos rápidos, casos de usuario convertidos en clases y funciones, y diseño de API.

Diseño y desarrollo

Código reutilizable, modelos de diseño, optimización y traducción entre lenguajes.

Pruebas y calidad

Pruebas unitarias, datos de prueba, casos límite, aserciones, canalizaciones CI/CD y puertas de calidad.

Implementación

Archivos de configuración por entorno, scripts de implementación y documentación actualizada.

Soporte y mantenimiento

Corrección de errores, refactorización, documentación sincronizada y comprensión de código heredado.

LIMITACIONES

Qué tener en cuenta al adoptar la IA

Calidad y corrección

- Puede sugerir código con errores
- No siempre sigue las prácticas de seguridad
- Puede interpretar mal el contexto amplio

Lenguajes y marcos

- Rendimiento variable según el lenguaje
- Menor precisión en tecnologías de nicho o recientes

Datos de entrenamiento

- Posible sesgo o prácticas obsoletas
- Debate abierto sobre derechos de autor

Problemas complejos

- Limitado en diseño arquitectónico de alto nivel
- No reemplaza la creatividad humana

API REST de métricas de uso de GitHub Copilot

Acceso a métricas para miembros de empresa, equipos y organizaciones: uso diario, finalizaciones, interacciones de chat, participación, editores y lenguajes, sugerencias, aceptaciones y usuarios activos.

Empresa

GET /enterprises/{enterprise}/copilot/usage

Equipo empresarial

GET /enterprises/{enterprise}/team/{team_slug}/copilot/usage

Organización

GET /orgs/{org}/copilot/usage

Respuesta: 200 OK con una matriz JSON de métricas diarias, desglosada por editor e idioma. Autenticación con token Bearer.

Cuatro fases para evaluar el impacto con datos

1

Evaluación

Satisfacción del desarrollador y finalización de tareas; usuarios activos diarios, tasa de aceptación y líneas de código aceptadas.

2

Adopción

Supervisar productividad y habilitación a medida que Copilot se integra, e identificar necesidades de entrenamiento.

3

Optimización

Relacionar el impacto con objetivos organizativos: tiempo de comercialización y calidad del código.

4

Eficiencia sostenida

Evaluación continua y ajustes para garantizar mejoras de productividad a largo plazo.

Formatos, cadencia y análisis de resultados

Formato corto

- Comentarios frecuentes y rápidos, posible cada dos semanas.
- Satisfacción general, desafíos específicos y tiempo ahorrado.
- "¿Cómo se sentiría si ya no pudiera usar GitHub Copilot?"

Formato largo

- Análisis profundo en las fases de evaluación y adopción.
- No más de una vez cada cuatro semanas.
- Impacto en el equipo: mejores revisiones y combinación más rápida en producción.

Análisis: respuestas anonimizadas, seguimiento en herramientas de BI y análisis de tendencias para impulsar la mejora continua.

MEDICIÓN BASADA EN DATOS

Más allá de la evidencia anecdótica

La combinación de la API REST de métricas de uso y la encuesta de desarrolladores permite entender con datos concretos cómo GitHub Copilot influye en el proceso de codificación.

**Decisiones
fundamentadas**

**Impacto medible en
el flujo de trabajo**

**Áreas de
optimización**

Mejora continua